



TTK4147 Real-time Systems

02 Hardware and Operating Systems

27.08.2025

1

TTK4147 – 2025: Hardware and Operation Systems

 NTNU

1

Reference group students of TTK4147 2025

So far, the student's reference group consists of

- Konstantin Quillfeldt, konstanq@stud.ntnu.no

We still need a voluntary for the group

2

TTK4147 – 2025: Hardware and Operation Systems

 NTNU

2

1

Today

- Hardware
 - The components of a computer system and how the components make programs to execute.
- Operating Systems
 - Brief history of operating systems.
 - The roles of operating systems.
 - Multiprogramming operating systems.
 - Process

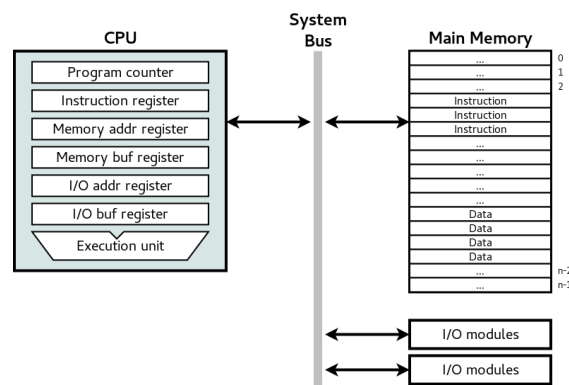
3

TTK4147 – 2025: Hardware and Operation Systems



3

Computer Components



The purpose of this lecture:
To get a good enough
understanding of the inner
workings of a computer
that you can understand
the topics that will be
covered later in the course.

The purpose of a computer:
To execute programs!

4

TTK4147 – 2025: Hardware and Operation Systems



2

4

Program execution

- A computer is made for executing programs, normally written in a **high-level language**, e.g. C:
- By use of a compiler, the C-code is transferred to a **computer-specific** number of **assembly instructions**:
- Each computer-specific assembly instructions is represented by a specific **binary value**, which is interpreted by the computer.
- The binary value can consist of an **instruction part** and a **pointer to an operand**

```
for(i=0;i<size;i++){
    a[i] = 0;
    fread(&a[i],sizeof(char),1,f);
    printf("0x%x\t\n",a[i] & 0xff);
}
```

Mnemonic(s)	Operation
ADD	Add memory to D accumulator
CMPL	Compare memory from D accumulator
EXG D, R	Exchange D with X, Y, S, U, or PC
LDD	Load D accumulator from memory
SEX	Sign Extend B accumulator into A accumulator
STD	Store D accumulator to memory
SUBD	Subtract memory from D accumulator
TFR D, R	Transfer D to X, Y, S, U, or PC
TFR R, D	Transfer X, Y, S, U, or PC to D

SUBA 245	80 245
CMPA 245	81 245
SBCA 245	82 245
SUBD 245	83 245
ANDA 245	84 245
BITA 245	85 245

How does an assembly instruction work?

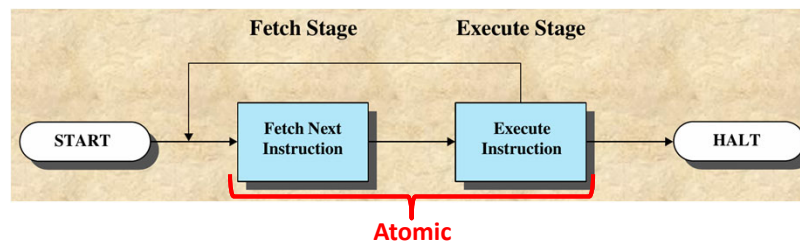
5

TTK4147 – 2025: Hardware and Operation Systems

NTNU

Microprocessor Instructions

- A program executed by a microprocessor consists of a set of instructions.



- In its simplest form, instruction execution consist of two steps
 - The «Fetch Stage» loads the instruction from memory into the instruction register (IR), according to the program counter (PC).
 - The «Execute State» performs the fetched instruction.
 - The Fetch and Execute Stages form one single atomic unit

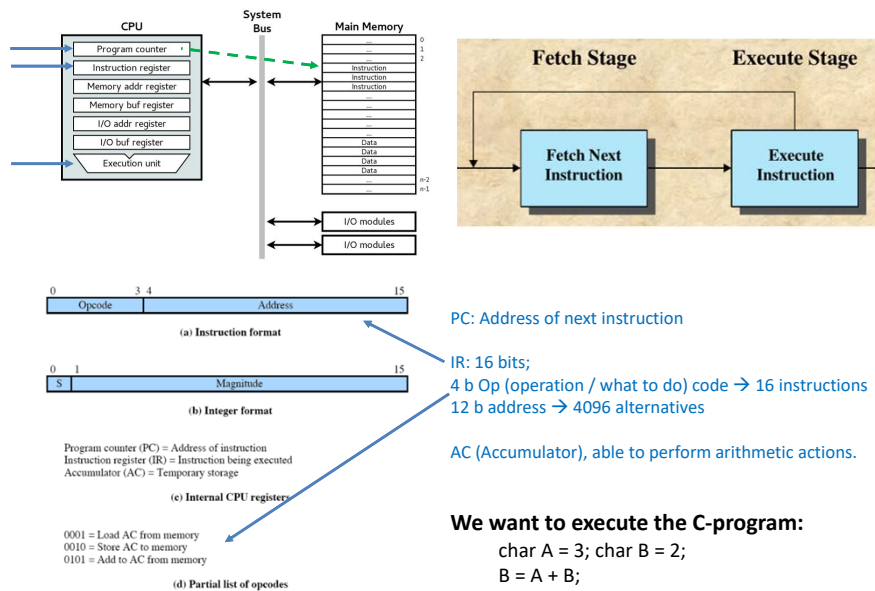
6

TTK4147 – 2025: Hardware and Operation Systems

NTNU

3

Simple Example of program execution



7

TTK4147 – 2025: Hardware and Operation Systems



7

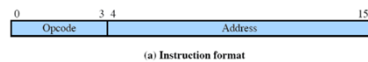
Simple Example of program execution (1/6)

Start condition:

Program counter = 300:

First fetch stage

- The content of 0x300 is transferred to IR

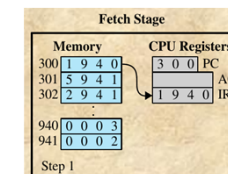


Program counter (PC) = Address of instruction
Instruction register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

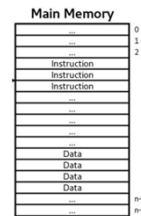
(c) Internal CPU registers

0001 = Load AC from memory
0010 = Store AC to memory
0101 = Add to AC from memory

(d) Partial list of opcodes



Remember



PC: Address of next instruction

IR: 16 bits;
4 b Op (operation / what to do) code → 16 instructions
12 b address → 4096 alternatives

AC (Accumulator), able to perform arithmetic actions

B = A + B
Previous cond.: A = 3, B = 2
A in location 940, B in 941

8

TTK4147 – 2025: Hardware and Operation Systems



4

8

Simple Example (2/6)

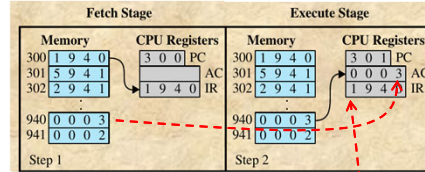
First execute stage

- The instruction in IR is executed, which is to transfer the content of 0x940 into AC (accumulator register)
- PC has been incremented (default action)

$$B = A + B$$

Previous cond.: $A = 3, B = 2$

A in location 940, B in 941



(a) Instruction format



(b) Integer format

Program counter (PC) = Address of instruction
Instruction register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from memory
0010 = Store AC to memory
0101 = Add to AC from memory

(d) Partial list of opcodes

9

TTK4147 – 2025: Hardware and Operation Systems



9

Simple Example (3/6)

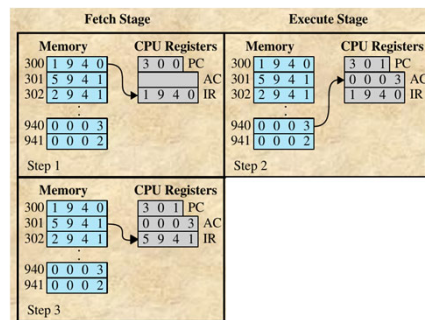
Second fetch stage

- The content of 0x301 is transferred to IR

$$B = A + B$$

Previous cond.: $A = 3, B = 2$

A in location 940, B in 941



(a) Instruction format



(b) Integer format

Program counter (PC) = Address of instruction
Instruction register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from memory
0010 = Store AC to memory
0101 = Add to AC from memory

(d) Partial list of opcodes

10

TTK4147 – 2025: Hardware and Operation Systems



5

10

Simple Example (4/6)

$$B = A + B$$

Previous cond.: $A = 3, B = 2$

A in location 940, B in 941

Second execute stage

- The instruction in IR is executed, which adds 0x941 to the AC
- PC has been incremented (default action)



(a) Instruction format



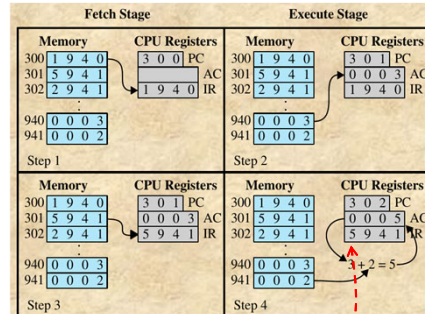
(b) Integer format

Program counter (PC) = Address of instruction
Instruction register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from memory
0010 = Store AC to memory
0101 = Add to AC from memory

(d) Partial list of opcodes



11

TTK4147 – 2025: Hardware and Operation Systems



Simple Example (5/6)

$$B = A + B$$

Previous cond.: $A = 3, B = 2$

A in location 940, B in 941

Third fetch stage

- The content of 0x302 is transferred to IR



(a) Instruction format



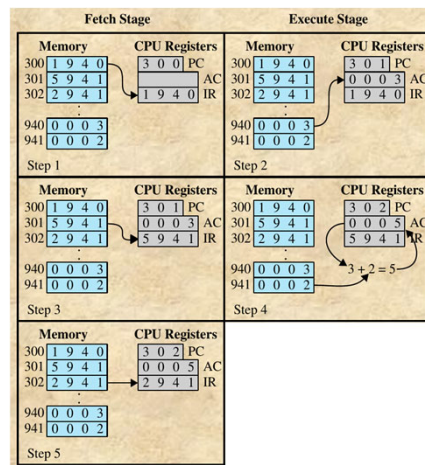
(b) Integer format

Program counter (PC) = Address of instruction
Instruction register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from memory
0010 = Store AC to memory
0101 = Add to AC from memory

(d) Partial list of opcodes



12

TTK4147 – 2025: Hardware and Operation Systems



Simple Example (6/6)

Third execute stage

- The instruction in IR is executed, which store the AC to address 0x941
- PC has been incremented (default action)



(a) Instruction format



(b) Integer format

Program counter (PC) = Address of instruction
Instruction register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from memory
0010 = Store AC to memory
0101 = Add to AC from memory

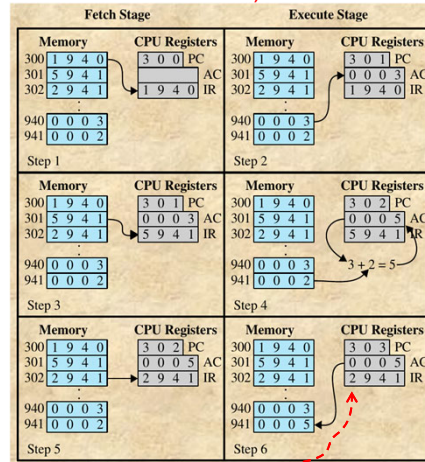
(d) Partial list of opcodes

$B = A + B$

Previous cond.: $A = 3, B = 2$

A in location 940, B in 941

New cond.: $A = 3, B = 5$



13

TTK4147 – 2025: Hardware and Operation Systems

NTNU

13

C code execution, in short

The C statement

$B = A + B;$

is executed by running the assembly code

(A is located in memory location 940, B in 941)

load AC from 940

add to AC from 941

stor AC to 941

14

TTK4147 – 2025: Hardware and Operation Systems

NTNU

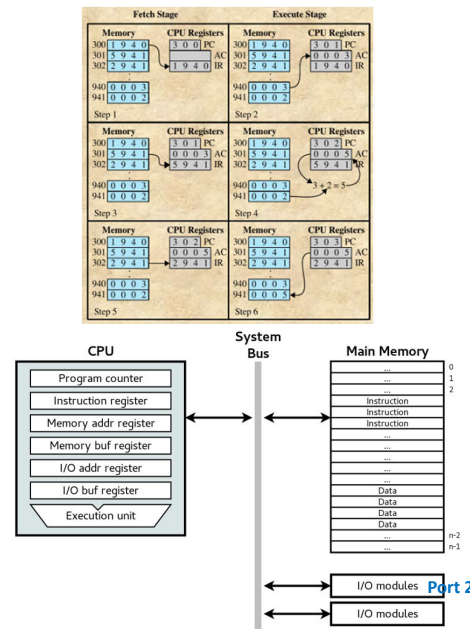
14

7

Instruction Categories

Instructions will typically fall into four categories:

- **Processor $\leftrightarrow$ memory:** Instructions for transfer data between processor and memory.
(E.g. Load AC from memory)
- **Processor $\leftrightarrow$ I/O:** Instructions for transferring data between processor and peripheral devices through an I/O module.
(E.g. Load AC from port 2)
- **Data processing:** Instructions for doing arithmetic or logical operations on data.
(E.g. Add to AC from memory)
- **Control:** Instructions that alter the sequence of execution.
(E.g. Jump to location xx)



15

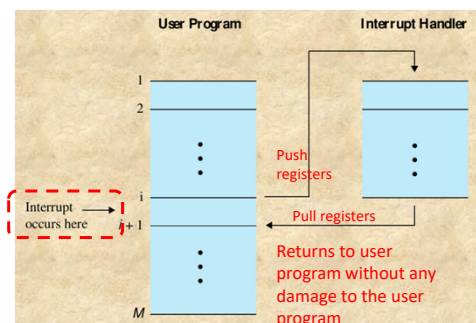
TTK4147 – 2025: Hardware and Operation Systems

NTNU

15

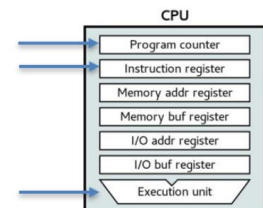
Interrupts (Norw: Avbrudd)

- It is a mechanism for interrupting the normal sequence of the processor's execution.
- Intended to improve processor utilization.



Without any damage to the user program?

- Have to push (push to memory) registers to be used by the interrupt routine before entering this
- Have to restore (pull from memory) the copied registers when leaving the interrupt routine



16

TTK4147 – 2025: Hardware and Operation Systems

NTNU

16

8

Interrupts can be grouped into four classes

- **Program:** Generated by some condition that is a result from an instruction execution. Often called software interrupt.
Two groups: Exception (unplanned, e.g. divide by zero) and normal sw interrupt (e.g. SWI, SWI2)
- **Timer:** Generated by a timer within the processor that can be programmed to trigger at regular interval or after a certain amount of time has passed.
Gives the possibility to synchronize the program with time.
- **I/O:** Generated by an I/O controller to signal completion of an operation or some other condition, such as an external event has happen.
Gives the possibility to synchronize the program with external events.
- **Hardware failure:** Generated by a failure in the hardware. (E.g. Memory parity error)

17

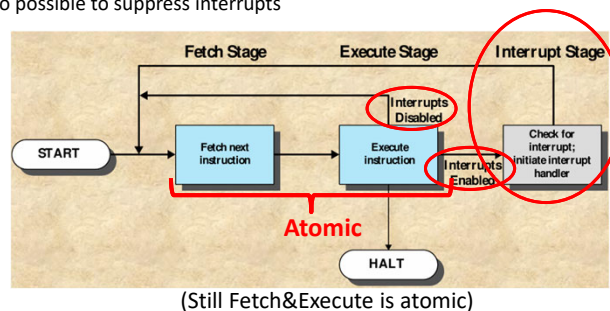
TTK4147 – 2025: Hardware and Operation Systems

NTNU

17

Interrupts effect on Instruction Execution

- Need to add an interrupt stage to the previously mentioned fetch and execute states.
 - In the interrupt stage, the processor checks if there is an interrupt signal.
 - If there is an interrupt, then the processor suspend the current execution and executes an interrupt-handler routine (typically part of the OS).
 - If there is no interrupt, then the processor will go back to the fetch stage.
 - Also possible to suppress interrupts



- Some overhead (interrupt-handler routine) when an interrupt occur.
 - Negligible compared to the potential benefits.
 - No overhead when no interrupt occurs (apart for complexity within the processor).

18

TTK4147 – 2025: Hardware and Operation Systems

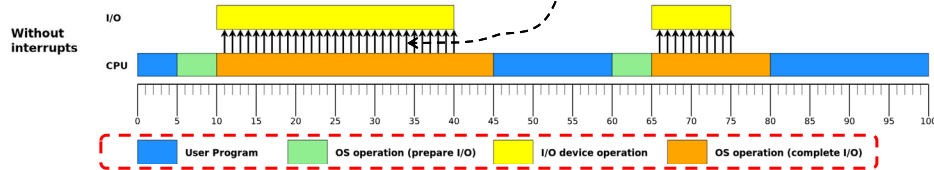
NTNU

18

9

Interrupt Example

- A simple example with a user program that do two I/O operations. The I/O is HW driven.
 - First I/O operation is relatively long.
 - The second is shorter.
 - Each I/O operation requires the OS running on the CPU to do some preparation and completion work.
- Without interrupts, the I/O completion block continuously poll the I/O device to check if it is complete.
 - The CPU is unable to do anything else during this period, just polling.



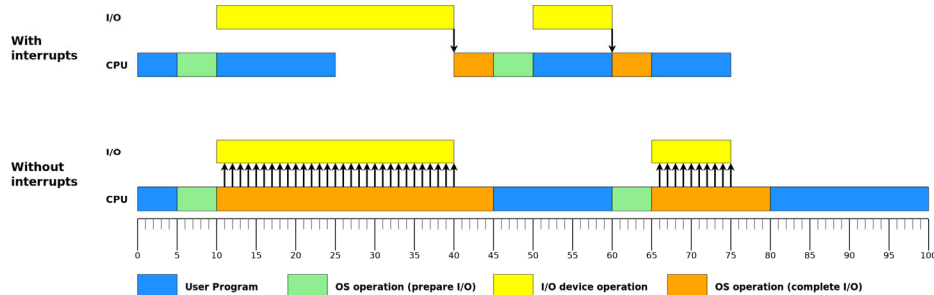
19

TTK4147 – 2025: Hardware and Operation Systems

NTNU

Interrupt Example continue

- With interrupts, the user program can continue, and it will be interrupted when the I/O completes.
 - In the first (long) I/O operation, the user program segment is completed while the I/O device perform its operation.
 - In the second (short) I/O operation, the user program starts, but is then interrupted. It continues after the I/O completion block is done.



20

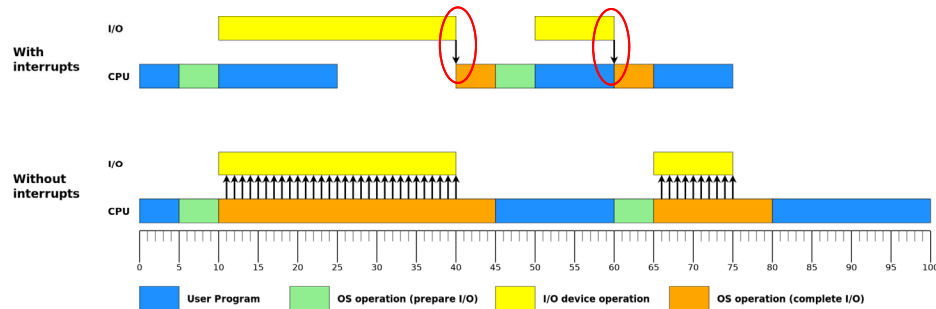
TTK4147 – 2025: Hardware and Operation Systems

NTNU

10

Interrupt Example continue

- By increasing the «smartness» of the HW (giving the HW ability to notify, interrupt, when it is finished), the CPU no longer need to continuously check whether the HW is finished
- Interrupts **increase** the amount of time the CPU can do useful work.
- Interrupts **decrease** the amount of time (Real time) needed for the program, as the program doesn't need to continuously check whether the HW is finished.



21

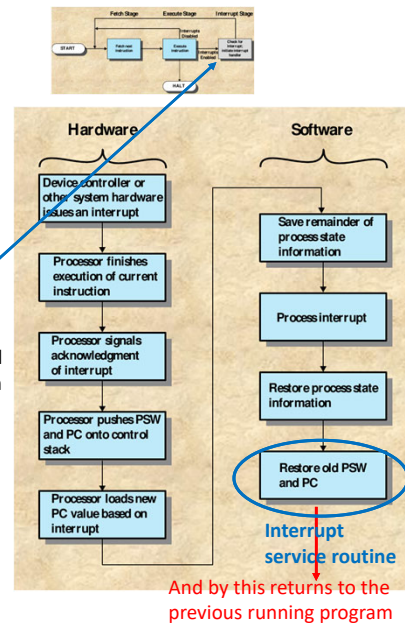
TTK4147 – 2025: Hardware and Operation Systems

NTNU

21

Interrupt Processing

- Interrupts are handled partly by software and partly by hardware.
- The hardware:
 - A device, the program or other hardware issue an interrupt to the CPU.
 - The CPU do the fetch and execute the current instruction.
 - Test for interrupts. If any is found, the CPU sends an acknowledge of it.
 - CPU save the (minimal) information needed for resuming the currently running program (PSW and PC).
 - CPU loads new PC that points to the interrupt-handling routine for the specific interrupt that was received.
- If you use a microcontroller without OS, then you will have to program some of the rest. (Such as priority handling, standard drivers..)



PSW = Program Status Word, status set as a result of previous executed instruction

22

TTK4147 – 2025: Hardware and Operation Systems

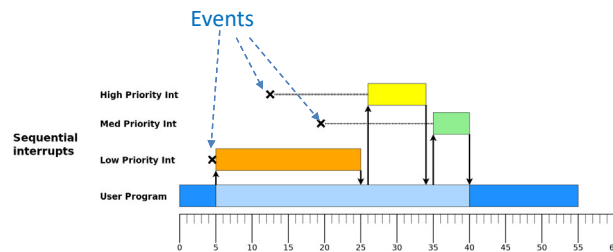
NTNU

22

11

Multiple Interrupts

- What will happen if an interrupt occur while another interrupt is processing?
- One (simple) approach is to disable interrupts while one interrupt is being processed.
 - Called sequential interrupt processing.
 - If an interrupt occurs while interrupts are disabled, it will be ignored until interrupts are enabled again.
 - Drawback is that a highly important interrupt can be delayed by a less important interrupt.



23

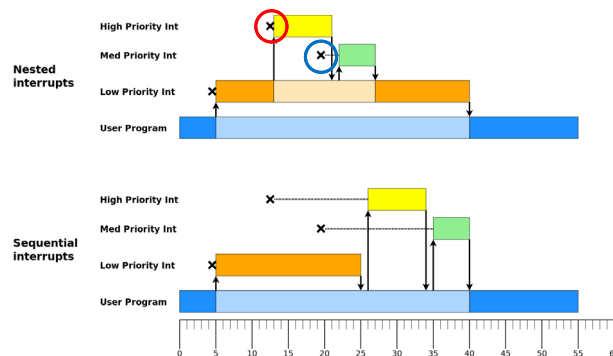
TTK4147 – 2025: Hardware and Operation Systems



23

Multiple Interrupts

- Second approach is to have priorities for interrupts and the ability to "interrupt an interrupt".
 - Called nested interrupts.
 - Guarantees that higher priority interrupts gets handled first.
 - A **higher priority interrupt will replace a lower priority interrupt**, while a **lower priority interrupt will not replace a higher priority interrupt**



24

TTK4147 – 2025: Hardware and Operation Systems

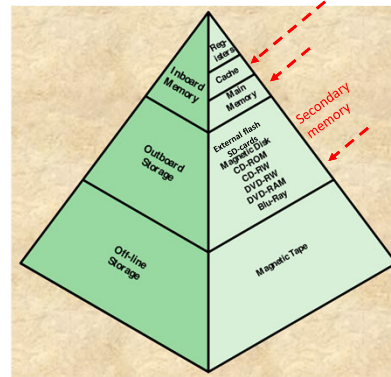


12

24

Memory Hierarchy

- There is a trade-off between three characteristics of different types of memory/storage:
 - Capacity
 - Access time
 - Cost
- High capacity and low access time both drives cost
- Memory at the top of the pyramid is accessed frequently.
 - Need low access time
 - Low capacity as cost per bit is high
- Memory further down is accessed less frequently.
 - Higher capacities are needed
 - Longer access time as less expensive memory is used.



25

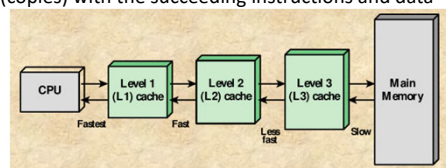
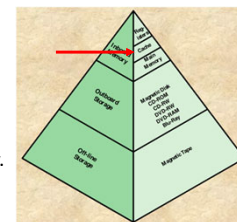
TTK4147 – 2025: Hardware and Operation Systems



25

Cache Memory

- All processor instructions fetch instruction from memory (and often other memory content).
 - CPU is much faster than main memory.
 - Would not be possible to fully utilize the CPU.
- Solution = Cache
 - Small and very fast memory between CPU and main memory.
 - Invisible to the OS (just works)
 - Can have several levels.
 - Fills up (copies) with the succeeding instructions and data



- Principle of Locality, why do we pre-fetch instructions?
 - Memory references by the processor tend to cluster.
 - If you need one instruction, chances are that you soon will need the next 10.

26

TTK4147 – 2025: Hardware and Operation Systems



13

26

Cache Implementation

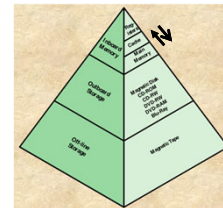
(Allocation and write back)

- Memory mapping function determines where to store memory blocks in cache.
 - When one block is stored in cache, it replace an existing block.
 - Which block is least likely to be used again soon?
 - Flexible mapping function require complex circuitry to use the cache.
 - Least recently used (LRU) algorithm is often used, which replace the block that haven't been reference for the longest time.

- Cashe mirrors the memory, both program and data.

The CPU changes the data. Write policy determines how often the changes in cache is written back to the main memory.

- One extreme: Update main memory each time cache block value is changed.
 - Many write operations.
- Other extreme: Only update main memory when the cache block value is replaced.
 - Minimal write operations.
 - Leaves the main memory in an obsolete state.



27

TTK4147 – 2025: Hardware and Operation Systems



27

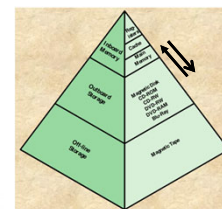
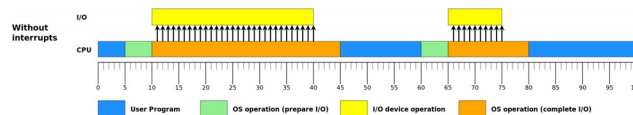
I/O Operation Techniques. How to move data between storages

How to transfer large amount of data from I/O device (hard drive) and main memory?

3 levels of smartness:

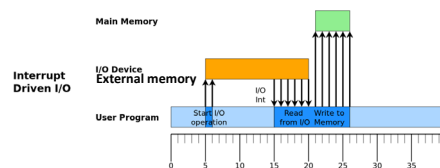
1. Programmed I/O

- The CPU send command to I/O device to order data and poll it periodically for data to be ready for transferring to memory.
- CPU is busy for all or most of the duration of the I/O operation.



2. Interrupt driven I/O

- Use interrupts instead of polling.
- Inefficient for each byte of data to be read by the CPU and then written to the memory, useful for cluster of data.



28

TTK4147 – 2025: Hardware and Operation Systems



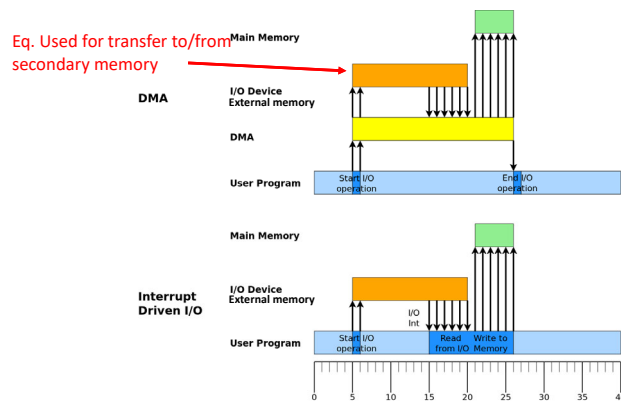
14

28

I/O Operation Techniques (continue)

3. Direct Memory Access (DMA) for moving bulk of data

- Interaction with the I/O device is “outsourced” to a separate DMA (Direct Memory Access) module (HW).
- CPU is only informed when the transfer is complete and the requested information is located in memory.



29

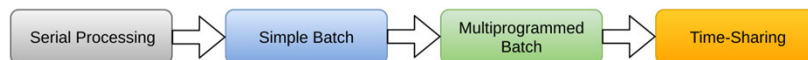
TTK4147 – 2025: Hardware and Operation Systems

NTNU

29

Evolution of Operating Systems

Time line



- No OS
- Manually load instructions
- Results on printer
- Hardcopy sign-up sheet

```

$JOB
$FTN
.
$LOAD
$RUN
.
$END
  
```

Read one record from file	15 μ s
Execute 100 instructions	1 μ s
Write one record to file	15 μ s
TOTAL	31 μ s

$$\text{Percent CPU Utilization} = \frac{1}{31} = 0.032 = 3.2\%$$

CPU often idle

30

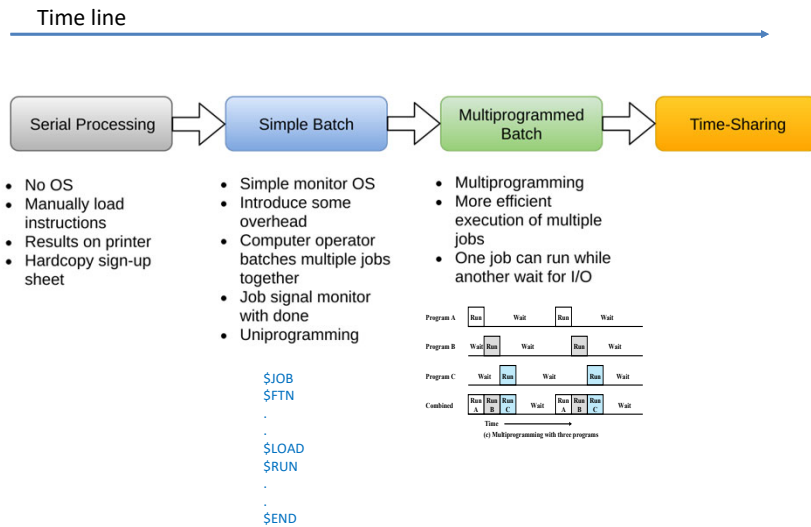
TTK4147 – 2025: Hardware and Operation Systems

NTNU

30

15

Evolution of Operating Systems



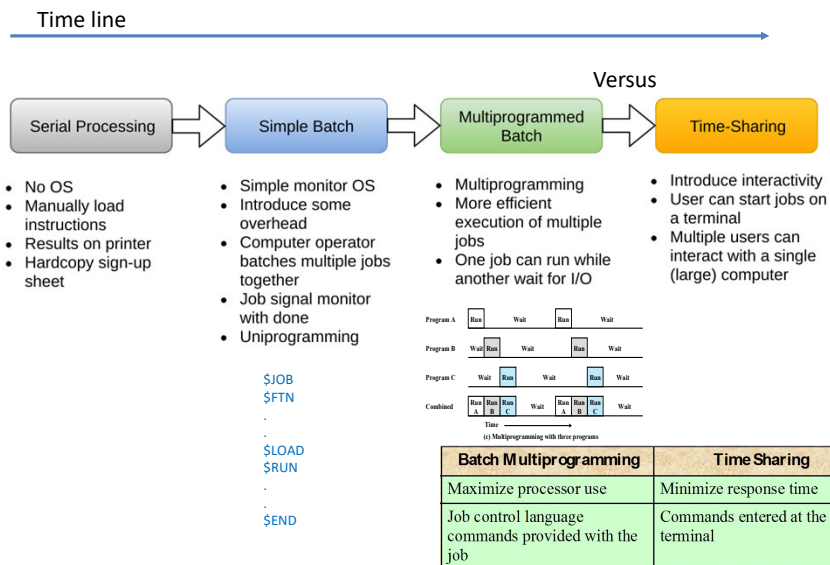
31

TTK4147 – 2025: Hardware and Operation Systems



31

Evolution of Operating Systems



32

TTK4147 – 2025: Hardware and Operation Systems



16

32

Operating Systems

Questions from last week:

- Why are operating systems important for real-time?
- Why do we need an operating system at all? (Or do we really?)

Today we will at least give some answers to these questions.

The Stallings book describe three reasons for having an operating system

- Convenience
- Efficiency
- Ability to evolve

33

TTK4147 – 2025: Hardware and Operation Systems

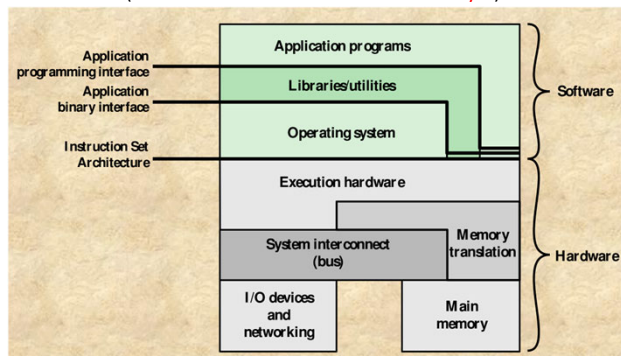


33

Convenience

The operating system as a user/computer interface.

- The operating system can be seen as a layer between the computer hardware and the application program (user or developer).
 - The OS developers have written the code to interact with the hardware so you do not have to do that...
 - Instead of interacting directly with the hardware, you interact with services provided by the OS. (Creates a “standardized” abstraction layer)



34

TTK4147 – 2025: Hardware and Operation Systems

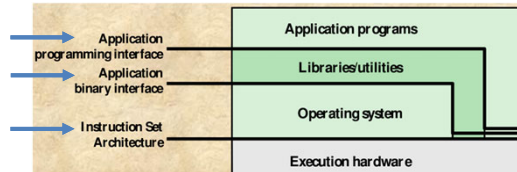


17

34

Convenience - Interfaces

- Instruction set architecture (ISA)
 - **Hardware <--> OS / software**
 - Defined by hardware / CPU
 - Machine language instructions that a computer can perform.
 - User programs (or libraries) can sometimes bypass the OS and use the ISA directly (low level programming).
- Application binary interface (ABI)
 - **OS <--> Libraries**
 - User programs and libraries can access OS and hardware (ISA) services through system calls.
- Application programming interface (API)
 - **Application <--> Library**
 - Libraries usually comes with an OS (but not strictly part of the OS).
 - Provide functions that other user programs can use to access the OS and hardware through system calls and other commonly used functionality.



0001 = Load AC from memory
 0010 = Store AC to memory
 0101 = Add to AC from memory

35

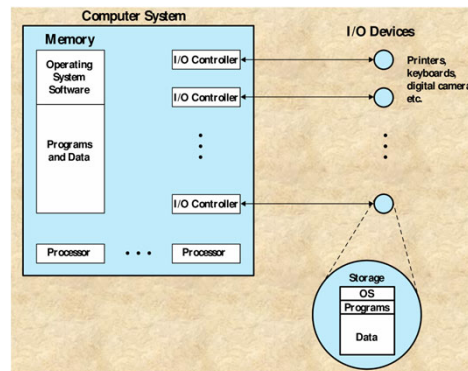
TTK4147 – 2025: Hardware and Operation Systems



35

Efficiency

- The OS allow efficient use of the computer system resources.
 - Schedule when different programs can run on the CPU
 - Control the memory of the system, allocate memory for the user programs.
 - Access I/O resources.
 - **Utilize the processor**



36

TTK4147 – 2025: Hardware and Operation Systems



18

36

Ability to Evolve

- Hardware constantly evolve
 - An OS has to be able to use upgraded hardware
 - ... and new types of hardware.
- Allow for development of new services
 - An OS should make it convenient for developers to make new programs and services for the system.
- All complex software systems (as an OS) has software faults or bugs.
 - An OS has to allow the OS to be upgraded.
 - Fine if also programs and applications can be updated.

Give ability to Evolve

- **without changing the application program**
- **by keeping backwards compatibility**

37

TTK4147 – 2025: Hardware and Operation Systems



37

Multiprogramming

What is multiprogramming?

- The ability for the computer to (seemingly) do multiple tasks in parallel.
- A (single processor) computer can not perform tasks in parallel, but can do tasks sequentially really, really fast...



How computers multitask



How humans multitask



38

TTK4147 – 2025: Hardware and Operation Systems



38

19

Uniprocessing vs Multiprocessing

3 different jobs with different requirements

	JOB1	JOB2	JOB3
Type of job	Heavy compute	Heavy I/O	Heavy I/O
Duration	5 min	15 min	10 min
Memory required	50 M	100 M	75 M
Need disk?	No	No	Yes
Need terminal?	No	Yes	No
Need printer?	No	No	Yes

39

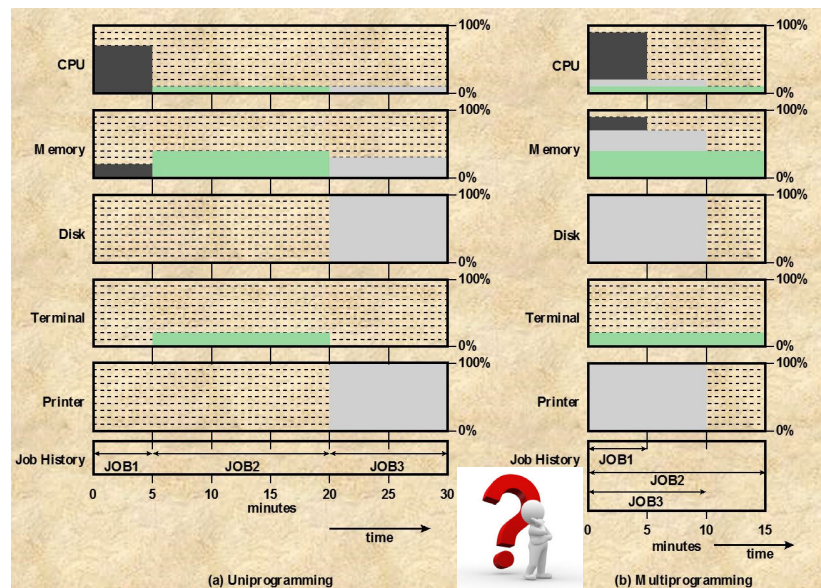
TTK4147 – 2025: Hardware and Operation Systems

NTNU

39

Utilization Histogram

What is the assumption?



40

TTK4147 – 2025: Hardware and Operation Systems

NTNU

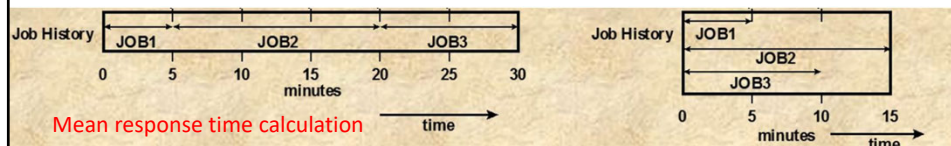
20

40

Uniprocessing vs Multiprocessing

Effect of Multiprogramming on Resource Utilization

	Uniprogramming	Multiprogramming
Processor use	20%	40%
Memory use	33%	67%
Disk use	33%	67%
Printer use	33%	67%
Elapsed time	30 min	15 min
Throughput	6 jobs/hr	12 jobs/hr
Mean response time	18 min	10 min



41

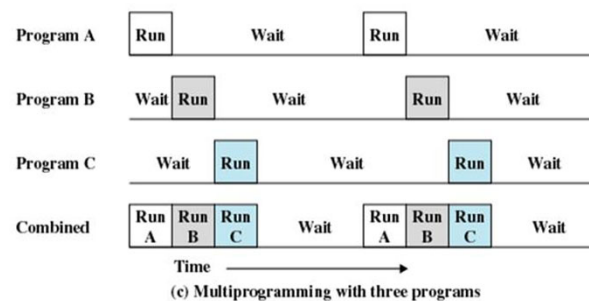
TTK4147 – 2025: Hardware and Operation Systems

NTNU

41

Multiprogramming Operating System

- How is multiprogramming implemented by Operating Systems?



- Operating systems need a mechanism for running several tasks, and switch between these, for:
 - To maximize processor utilization.
 - Providing reasonable response times.
 - Let different tasks use different resources.
 - Allow tasks to communicate with each others.

42

TTK4147 – 2025: Hardware and Operation Systems

NTNU

42

21

Do we need an OS?

Using bare metal programming

Bare metal (or bare machine) refers to a computer executing instructions (high level language or assembly instructions) directly on logic hardware without an intervening operating system.

Today mostly applicable to embedded systems generally with time-critical latency requirements and / or for to minimize the code.

E.g.,

Use of the construction “Big While Loop” can give a primitive runtime system providing priority to the different jobs

```
while ( 1) {
Top:   if ( Flag-A ) { gosub-A(); Flag-A=0; goto Top;}
        if ( Flag-B ) { gosub-B(); Flag-B=0; goto Top;}
        if ( Flag-C ) { gosub-C(); Flag-C=0;}
        if ( Flag-D ) { gosub-D(); Flag-D=0;}
}
```

Flag-A ... Flag-D are set by interrupt routines.
Be aware of the priority mechanism.

43

TTK4147 – 2025: Hardware and Operation Systems



43

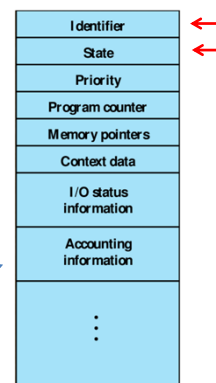
Process

What is a process?

A process is an entity that can be assigned to and executed by a processor.

Consists of:

- The program code that defines what the process does.
 - Basically the source code you write.
 - Shared between all processes that execute the same program.
- The set of data associated with the process.
 - Each process has their own set of data.
 - Stored in a process control block data structure.
 - OS has to manage the data structures of the different processes.



44

TTK4147 – 2025: Hardware and Operation Systems



22

44

Process Image

How to describe a process:

- User Data
 - The modifiable part of the user space. May include program data, a user stack area, and programs that may be modified.
- User Program
 - The program to be executed.
- Stack
 - Each process has one or more last-in-first-out (LIFO) stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls.
- Process Control Block
 - Data needed by the OS to control the process.



Identifier
State
Priority
Program counter
Memory pointers
Context data
I/O status information
Accounting information
⋮

45

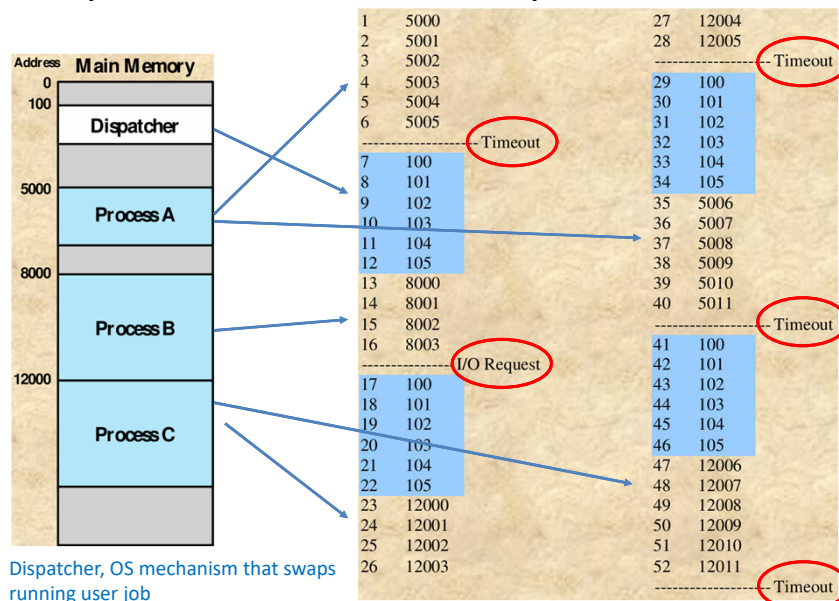
TTK4147 – 2025: Hardware and Operation Systems



45

Simple Multi Process Execution Example

What activates the dispatcher?



46

TTK4147 – 2025: Hardware and Operation Systems

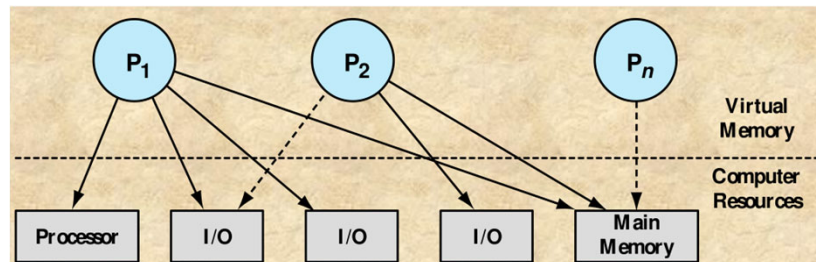


46

23

Process Access to Resources

- Processes exist and will currently have access to different computer resources.
 - P₁ is currently running, exists in main memory and has access to two I/O devices.
 - P₂ is currently ready and waits for access to an I/O device. It exists in main memory and has access to one I/O device.
 - P_n is currently suspended to hard-drive.



47

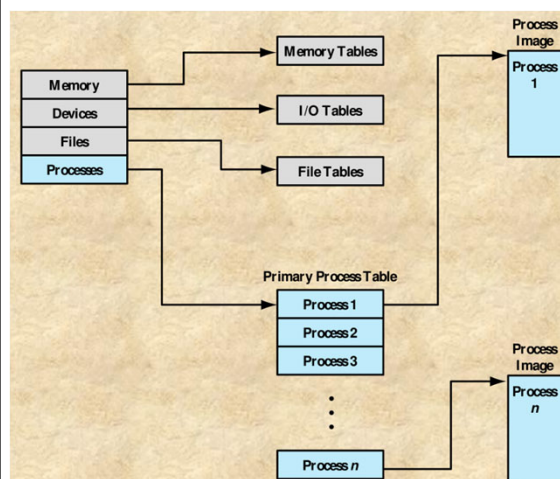
TTK4147 – 2025: Hardware and Operation Systems

NTNU

47

Operating System Control Structures

To administrate it's job, the OS needs an OS control structure (tables and lists)



- OS has to keep track of processes and resources.
 - How memory is allocated
 - Which I/O resources are used and their status
 - Information about files (often handled by file system)
 - Table with (process) image for all processes.
- These tables have cross-references to each others.

48

TTK4147 – 2025: Hardware and Operation Systems

NTNU

48

24



49

TTK4147 – 2025: Hardware and Operation Systems



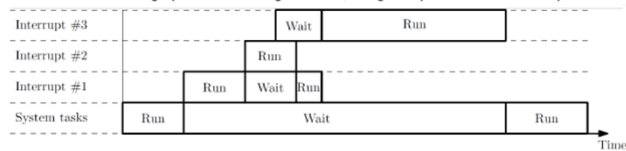
49

Endings

2022-01

Interrupt priorities from runtime graph

Based on the runtime graph show in the figure below, assign the priorities to the interrupts.

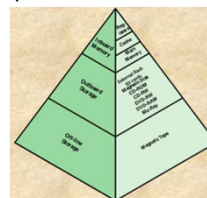


Which interrupt have Low, Medium or High priority **L:#3, M:#1, H#2**

0000-59

On most modern hardware platforms, physical memory is structured hierarchically. List the major levels in this hierarchy and explain what the trade-off is between these levels.

Capacity
Access time
Cost



50

TTK4147 – 2025: Hardware and Operation Systems



50

25